

Documentação de Sistema

Hotel Descanso Garantido

Autores:

Kenderson Santana Dias

Victor Gomes Barreto

Belo Horizonte
dezembro, 2025.

1 Documentação do Sistema

1 Documentação do Sistema.....	2
Introdução.....	3
Escopo do Sistema.....	3
Nome do sistema e de seus componentes principais.....	3
Missão ou objetivo do sistema.....	3
Arquitetura do Sistema.....	3
Funcionalidades do produto.....	4
Usuários e sistemas externo.....	4
Descrição.....	4
Documentação do código.....	5
Documentação da Estrutura de dados geral do software.....	5
Função cadastrarQuarto.....	5
Função listarQuartos.....	5
Função cadastrarCliente.....	5
Função pesquisarCliente.....	6
Função cadastrarEstadia.....	6
Função excluirEstadia.....	6
Função salvarBackup.....	7
Função carregarBackup.....	7
Testes do sistema.....	8
Casos de testes do sistema: função cadastrarQuarto.....	8
Casos de testes do sistema: função cadastrarEstadia.....	8

Introdução

Escopo do Sistema

Nome do sistema e de seus componentes principais

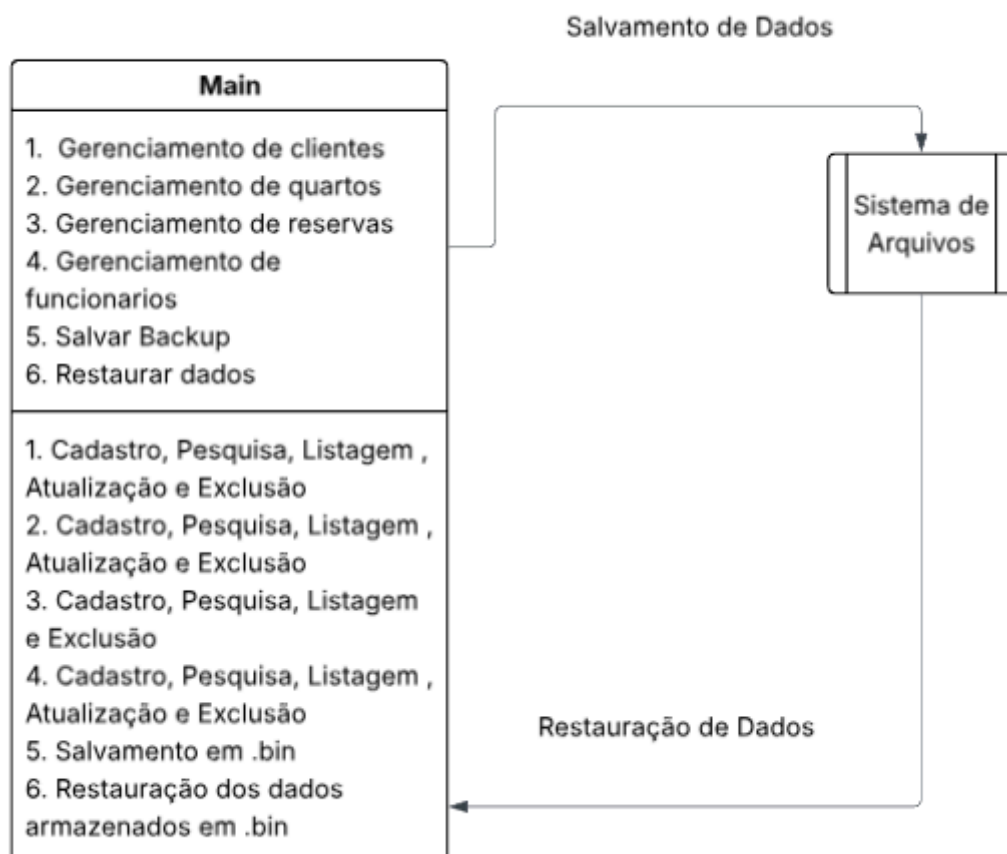
Hotel Descanso Garantido:

- *Hotel.cpp*

Missão ou objetivo do sistema

Gerenciar as operações fundamentais de um hotel, permitindo o cadastro e controle de clientes, funcionários, quartos e reservas (estadias), além de garantir a persistência dos dados através de sistema de backup.

Arquitetura do Sistema



O sistema é estruturado sobre quatro classes principais que interagem entre si para gerenciar as regras de negócio do hotel, além de uma camada de persistência de dados.

- **Quarto:** Responsável por armazenar as características físicas da acomodação (número, capacidade de hóspedes, valor da diária) e seu estado atual (Ocupado/Desocupado). É manipulado através de um array global `listaQuartos`.
- **Cliente:** Gerencia as informações pessoais dos hóspedes. Cada cliente possui um ID único que é utilizado para vincular reservas.
- **Funcionario:** Segue uma estrutura similar à de Clientes, armazenando dados da equipe, incluindo cargo e salário.
- **Estadia:** Atua como a classe associativa que conecta um **Cliente** a um **Quarto** por um período de tempo determinado. Ela calcula a quantidade de diárias baseada nas datas de entrada e saída e altera o status do quarto para "Ocupado".
- **Persistência (Backup):** O sistema não utiliza banco de dados relacional, mas sim manipulação direta de arquivos binários (`.dat`). As funções de backup serializam os objetos das classes e os gravam em disco, permitindo recuperar o estado do sistema após o reinício.

Funcionalidades do produto

Número	Funcionalidade do sistema
1	Cadastrar e gerenciar clientes
2	Cadastrar e gerenciar quartos
3	Cadastrar e gerenciar estadias (reservas)
4	Cadastrar e gerenciar funcionários
5	Salvar os dados em arquivos binários
6	Carregar os dados de arquivos binários

Usuários e sistemas externo

Número	Usuários	Definição
1	Recepcionista	Usuário que opera o sistema via terminal para realizar cadastros e reservas.
2	Sistema de Arquivos	O sistema interage externamente criando arquivos <code>.dat</code> para persistência

Descrição

<i>Classe</i>	<i>Definição</i>
<i>Quarto</i>	<i>Representa as unidades de hospedagem, contendo número, capacidade, valor e status (Ocupado/Desocupado).</i>
<i>Cliente</i>	<i>Armazena dados pessoais dos hóspedes como ID, nome, endereço e telefone.</i>
<i>Funcionario</i>	<i>Gerencia os dados da equipe (ID, nome, endereço e telefone), incluindo cargo e salário.</i>
<i>Estadia</i>	<i>Vincula um Cliente a um Quarto por um período de tempo, calculando custos e gerando códigos de reserva.</i>

Documentação do código

Documentação da Estrutura de dados geral do software

O software utiliza Programação Orientada a Objetos (POO) em C++. As classes encapsulam os dados e possuem métodos "Getters" e "Setters" para validação. Os dados são armazenados em memória utilizando arrays globais com tamanho definido por constantes (**MAX_QUARTOS**, **MAX_CLIENTES**, etc.)

Procedimentos de Quartos

Função cadastrarQuarto

```
void cadastrarQuarto() {  
    if (totalQuartos >= MAX_QUARTOS) { ... }  
    // ... (lógica de input)  
    listaQuartos[totalQuartos].setNumDoQuarto(num);  
    // ...  
    totalQuartos++;  
}
```

Verifica se o limite de quartos foi atingido. Recebe do usuário o número, quantidade de hóspedes e valor da diária, definindo o status inicial como "Desocupado" e armazenando o objeto na lista global.

Função listarQuartos

Assinatura: `void listarQuartos()`

Parâmetros: Nenhum.

Retorno: `void`.

```
void listarQuartos() {  
    // ...  
    for (int i = 0; i < totalQuartos; i++) {  
        cout << "Quarto " << (i+1) << ": Numero " <<  
        listaQuartos[i].getNumDoQuarto() ...  
    }  
}
```

Percorre o array **listaQuartos** e imprime no console os detalhes de todos os quartos cadastrados (número, hóspedes, valor e status).

Procedimentos de Clientes

Função cadastrarCliente

```
void cadastrarCliente() {  
    // ...  
    listaClientes[totalClientes].setId(id);  
    listaClientes[totalClientes].setNome(nome);  
    // ...  
    totalClientes++;  
}
```

Recebe ID, nome, endereço e telefone. Instancia um objeto **Cliente**, preenche os dados através dos métodos *set* e o adiciona ao array **listaClientes**.

Função pesquisarCliente

Assinatura: **void** **pesquisarCliente()**

Parâmetros: Nenhum (Recebe **int id** via entrada do usuário **cin**).

Retorno: **void** (Imprime os dados se encontrado).

```
void pesquisarCliente() {
    int id;
    // ... (input id)
    for (int i = 0; i < totalClientes; i++) {
        if (listaClientes[i].getId() == id) {
            // ... (print dados)
            return;
        }
    }
    cout << "Cliente nao encontrado.\n";
}
```

Solicita um ID ao usuário e percorre linearmente a lista de clientes. Se encontrar correspondência, exibe os dados completos; caso contrário, informa que não foi encontrado.

Procedimentos de Estadia

Função cadastrarEstadia

```
void cadastrarEstadia() {
    // Validações de existência de clientes e quartos
    // Input de datas e cálculo de diárias
    // Busca quarto disponível com capacidade compatível
    // ...
    listaEstadias[totalEstadias].setCod(cod);
    // ...
    listaQuartos[indiceQuarto].setStatus("Ocupado");
    totalEstadias++;
}
```

Ela realiza múltiplas validações: verifica se existem clientes e quartos, valida se a data de saída é posterior à entrada, busca quartos com status "Desocupado" e capacidade suficiente, calcula o valor total, gera um código de reserva e atualiza o status do quarto para "Ocupado".

Função excluirEstadia

Assinatura: **void** **excluirEstadia()**

Parâmetros: Nenhum (Recebe **char cod[]** via entrada do usuário).

Retorno: **void**.

```
void excluirEstadia() {
    // ...
    if (strcmp(listaEstadias[i].getCod(), cod) == 0) {
        // Libera o quarto (seta como "Desocupado")
    }
}
```

```
    // Remove a estadia do array  
    }  
}
```

Busca uma estadia pelo código. Ao excluir, o sistema automaticamente localiza o quarto associado e altera seu status de volta para "Desocupado", liberando-o para novas reservas.

Procedimentos de Backup

Função salvarBackup

Assinatura: `void salvarBackup()`

Parâmetros: Nenhum.

Retorno: `void`.

```
void salvarBackup() {  
    FILE *fp;  
    fp = fopen("quartos.dat", "wb");  
    // ... fwrite ...  
    // Repete para clientes, funcionarios e reservas  
}
```

Abre arquivos binários (`.dat`) em modo de escrita (`wb`) e grava o conteúdo integral dos arrays de objetos (`listaQuartos`, `listaClientes`, etc.) diretamente em um arquivo.

Função carregarBackup

Assinatura: `void carregarBackup()`

Parâmetros: Nenhum.

Retorno: `void`.

```
void carregarBackup() {  
    FILE *fp;  
    // Limpa contadores (totalQuartos = 0...)  
    fp = fopen("quartos.dat", "rb");  
    // ... fread ...  
}
```

Reseta os contadores do sistema e lê os arquivos binários, repopulando os arrays na memória RAM com os dados persistidos anteriormente.

Testes do sistema

Casos de testes do sistema: função cadastrarQuarto

Número	Variáveis de Entrada	Valores Válidos	Resultado Esperado	Valores Inválidos	Resultado Esperado
1	num, qtd, valor	101, 2, 150.00	Quarto cadastrado com sucesso na memória.	N/A (Validação feita nos Setters)	Se valor ≤ 0 , o setter mantém 0.0 ou valor anterior.
2	totalQuartos	< 1000	Cadastro permitido.	≥ 1000	Mensagem "Limite de quartos atingido!".

Casos de testes do sistema: função ListarQuarto

Número	Variáveis de Entrada	Valores Válidos	Resultado Esperado	Valores Inválidos	Resultado Esperado
1	totalQuartos	> 0	Lista todos os quartos cadastrados no console.	N/A	N/A.

Hotel Descanso Garantido

2	<i>totalQuartos</i>	<i>==0</i>	<i>Mensagem "Nenhum quarto cadastrado."</i>	<i>N/A</i>	<i>N/A".</i>
---	---------------------	------------	---	------------	--------------

Casos de testes do sistema: função cadastrarEstadia

<i>Número</i>	<i>Variáveis de Entrada</i>	<i>Valores Válidos</i>	<i>Resultado Esperado</i>	<i>Valores Inválidos</i>	<i>Resultado Esperado</i>
1	<i>idCliente</i>	<i>ID de cliente existente</i>	<i>Prossegue para dados da reserva.</i>	<i>ID inexistente</i>	<i>Mensagem "Cliente não encontrado".</i>
2	<i>qtdHospedes</i>	<i>Menor ou igual à capacidade do quarto</i>	<i>Lista quartos disponíveis.</i>	<i>Maior que a capacidade dos quartos</i>	<i>"Nenhum quarto disponível..".</i>
3	<i>dataEntrada, dataSaida</i>	<i>1010, 1510 (Saída > Entrada)</i>	<i>Calcula 5 diárias.</i>	<i>1510, 1010 (Saída <= Entrada)</i>	<i>Mensagem "Data de saída deve ser maior...".</i>

Casos de testes do sistema: função cadastrarCliente

Número	Variáveis de Entrada	Valores Válidos	Resultado Esperado	Valores Inválidos	Resultado Esperado
---------------	-----------------------------	------------------------	---------------------------	--------------------------	---------------------------

1	id	Inteiro > 0 e inédito	Cadastr o realizad o com sucesso.	id < 0	Mensag em "ID invalido! Digite um valor nao negativo ."
2	id	Inteiro único	Cadastr o realizad o.	id já existente na lista	Mensag em "ID ja existe! Tente outro."
3	totalClient es	< MAX_CLIE NTES	Permite o cadastro .	>= MAX_CLIE NTES	Mensag em "Limite de clientes atingido !"

Casos de testes do sistema: função pesquisarCliente

<i>Número</i>	<i>Variáveis de Entrada</i>	<i>Valores Válidos</i>	<i>Resultado Esperado</i>	<i>Valores Inválidos</i>	<i>Resultado Esperado</i>
1	id	ID existente no array	Exibe: ID, Nome, Endereço e Telefone.	ID inexistente	Mensagem "Cliente nao encontrado."

Casos de testes do sistema: função excluirEstadia

<i>Número</i>	<i>Variáveis de Entrada</i>	<i>Valores Válidos</i>	<i>Resultado Esperado</i>	<i>Valores Inválidos</i>	<i>Resultado Esperado</i>
<i>1</i>	<i>cod</i>	<i>Código existente (ex: "EST110")</i>	<i>Remove a estadia da lista E altera status do quarto para "Desocupado".</i>	<i>Código inexistente</i>	<i>Mensagem "Estadia não encontrada".</i>

Casos de testes do sistema: função cadastrarFuncionario

<i>Número</i>	<i>Variáveis de Entrada</i>	<i>Valores Válidos</i>	<i>Resultado Esperado</i>	<i>Valores Inválidos</i>	<i>Resultado Esperado</i>
<i>1</i>	<i>salario</i>	<i>Float > 0</i>	<i>Funcionário cadastrado.</i>	<i>salario <= 0</i>	<i>O setter mantém 0 ou ignora a alteração (conforme implementação da classe).</i>
<i>2</i>	<i>id</i>	<i>Inteiro único</i>	<i>Cadastro realizado.</i>	<i>id duplicado</i>	<i>Mensagem "ID já existe!".</i>

Casos de testes do sistema: funções de Backup

<i>Número</i>	<i>Variáveis de Entrada</i>	<i>Valores Válidos</i>	<i>Resultado Esperado</i>	<i>Valores Inválidos</i>	<i>Resultado Esperado</i>
---------------	-----------------------------	------------------------	---------------------------	--------------------------	---------------------------

1	<i>salvarBackup</i>	<i>Permissão de escrita no disco</i>	<i>Cria/Sobrescreve arquivos .dat e exibe "Backup salvo com sucesso".</i>	<i>Disco cheio ou sem permissão</i>	<i>Mensagem "Erro ao salvar..." (se ocorrer falha no fopen).</i>
2	<i>carregarBackup</i>	<i>Arquivos .dat existentes</i>	<i>Restaura os dados para a memória e exibe "Backup restaurado..."</i>	<i>Arquivos inexistentes</i>	<i>As listas permanecem vazias (ou com 0 itens) sem travar o sistema.</i>